



Cartesian Genetic Programming with a Modified Selection Operator for Combinational Circuit Design: Arithmetic Multipliers and Adders

Tomas Hulka, Radomil Matousek✉, Ladislav Dobrovsky,
Jakub Kudela, and Ondrej Hojny

Brno University of Technology, Brno, Czech Republic
{tomas.hulka,rmatousek,jakub.kudela}@vutbr.cz, dobrovsky@fme.vutbr.cz

Abstract. The evolutionary design of combinational logic circuits offers an innovative approach that often surpasses traditional methods, such as the Quine-McCluskey algorithm, in both efficiency and effectiveness. Cartesian Genetic Programming (CGP) emerges as a potent technique in this domain, enabling versatile circuit designs tailored to diverse requirements such as cost, gate count, and circuit speed. In this paper, we introduce an advanced modification of CGP, termed CGP-SA, which integrates the Simulated Annealing mechanism into the selection operator. This novel approach enhances the algorithm's ability to escape local optima, thereby fostering the discovery of more optimal solutions. We demonstrate the efficacy of CGP-SA through the design of three types of multipliers and two types of adders, utilizing diverse logic gate sets. This exploration not only reveals the flexibility of CGP-SA in handling various circuit design challenges but also highlights its adaptability to different optimization criteria.

Keywords: Cartesian Genetic Programming · Simulated Annealing · Digital Circuit · Circuit Design

1 Introduction

Digital multipliers and adders form a significant part of the arithmetic units in processors and computer architecture [8]. Especially the hardware implementation (FPGA, ASIC) of a digital signal processors (DSP), deep neural networks (DNN), processors' vector units (SIMD), and graphics processing units (GPU) requires a large number of multipliers [26]. Therefore it is important to optimize the multiplier circuit at the level of individual logical gates and transistors. The optimization of the number of logic gates and transistors is important not only to minimize the cost for mass production, but also to reduce the power consumption of the circuit or its delay [20].

Classic methods for the logic circuit minimization include Boolean algebra simplification, Karnaugh map and Quine-McCluskey algorithm. These methods

are very computationally intensive [24] and cannot be used for multicriterial optimization. Also they do not account for the number of transistors composing different gate types and require a correct starting solution [25].

Various bio-inspired optimization algorithms were recently applied to efficiently synthesize digital circuits' components such as adders and multipliers. These methods, which include genetic algorithm (GA) [2], particle swarm optimization (PSO) [19], differential evolution (DE) [27], simulated annealing (SA) [3], and harmony search (HS) [9], have proved their efficiency in solving various optimization problems. Every stage of design and manufacturing process of digital circuits depends on optimization methods. They can search for better components' parameters [4], their physical placement [13] or computing the masks for advanced lithography manufacturing process [28]. The natural extension of these algorithms is to evolve a structure and programs meeting a declared specification. However this expands the search space significantly. Such methods are genetic programming (GP) and grammatical evolution (GE). Cartesian genetic programming (CGP) is in particular a suitable method for evolving the structure of logical circuits [22].

In this work, we have built upon the foundational strengths of CGP, introducing a significant modification that integrates the simulated annealing technique. This modified CGP approach, which we term CGP-SA, transcends traditional limitations by allowing for the selection of offspring with lower fitness than their parents. This innovation not only broadens the potential solutions that can be explored but also enhances the algorithm's ability to escape local optima, a common challenge in evolutionary algorithms.

2 Digital Circuit

A digital circuit is a type of circuit that operates on different logic gates. The logic gates are used to differentiate between various power signals. It can be used to transmit the power signal to the different parts of the electronic device through various gates. This will help to create an output signal which is directly proportional to the energy level of the input signals. The three main ways of specifying the function of a combinational logic circuit are: Boolean algebra, truth table and logical block diagram. Typical logic gates are represented by boolean logic operators AND, OR, NOT, or more effectively by electronic implementation NOR or/and NAND, and also XOR, XNOR. Simple digital logic gates can be made by combining transistors, diodes and resistors. Inside integrated circuits the properties and availability of these components depend on the chosen manufacturing technology. For example Complementary metal-oxide-semiconductor (CMOS) uses symmetrical field effect transistor pairs as basic building block for logical gates. Because of this the NAND and NOR gates are easiest to implement requiring 4 transistors in two complementary pairs [1]. For required transistors counts for gate types see Table 1.

The *combinational logic circuit* is made up of logic gates whose output is determined by the present input only. The output does not depend on the pre-

vious outputs of the circuit. This type of circuit is widely used in the electronic industries for various purposes as multiplier in our case.

Table 1. Number of transistors for each gate type.

Gate	Num. of transistors
NOT	2
AND, OR	6
NAND, NOR	4
XOR, XNOR	9
(not x) AND y	8

Common combinational circuits made up from individual logic gates that carry In conclusion, the CGP-SA approach represents a significant step forward in the field of digital circuit design, offering a robust, efficient, and adaptable solution for complex problem-solving. The potential for future applications and enhancements of this method is substantial, promising exciting developments in both theoretical and practical aspects of circuit design and optimization.out a desired application include multiplier, multiplexers, encoders, decoders, full and half adders etc. Obviously, the more components (transistors) used to implement a particular digital circuit, the higher its cost will be. Another important consideration in optimizing a digital circuit may be the speed of the circuit or the electrical power consumption. An example of the 2-bit multiplier obtained by artificial evolution of CGP vs. the standard multiplier design is shown in Fig. 2.

3 Cartesian Genetic Programming

Cartesian Genetic Programming (CGP) was developed by Miller, J. in 1999 and 2000 from earlier cooperation with Thomson in 1997. The name ‘Cartesian’ means it represents a program using a two-dimensional grid of graph nodes (see Fig. 3 [7,14–16].

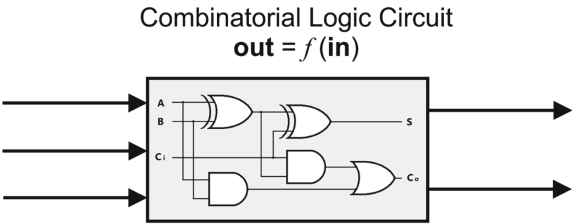


Fig. 1. A combinational logic circuit can have any number of digital inputs and any number of digital outputs.

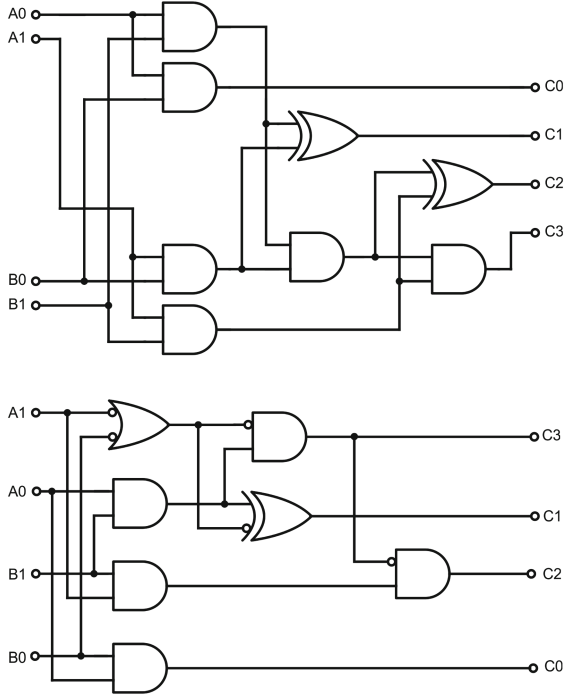


Fig. 2. Scheme of standard 2-bit binary multiplier circuit (above) and CGP optimized design (below).

The nodes in the grid are connected by oriented edges to form a directed acyclic graph. The number of columns and rows in the grid is set and pre-determined by the user. Nodes take their inputs in a feed-forward manner from either the output of nodes in l previous column or directly from a program input, where l is a user-defined parameter known as levels back [15] (Fig. 1).

The primary inputs and outputs of nodes are all labeled or addressed sequentially, with the inputs labeled 0 to $n - 1$ and outputs from 0 to m , and the nodes labeled immediately after the inputs (n onwards).

The nodes serve as the system's building blocks, for example, nodes can represent logic gates or arbitrary mathematical computations. The edges connect inputs and outputs of the blocks. The Genotype of CGP consists of integers denoting the type of blocks, which is called *function gene*, the remaining node genes say where the node gets its data from, thus it is called *connection genes*. This coding enables constant length of the genotype. Also it enables to express parallelism inherent to logical circuits [15].

When the genotype is transformed to phenotype, some nodes may not be connected and are ignored for the computation. Such disconnected nodes are referred as 'non-coding'. Therefore the fixed length of genotype can code variable length phenotype [14]. CGP uses a simple evolutionary algorithm called $1 + \lambda$,

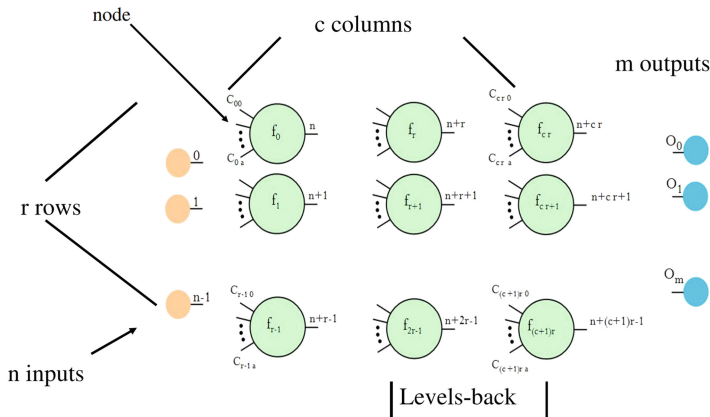


Fig. 3. Representation of CGP [14]

this algorithm has a population size of $1 + \lambda$, where λ is the number of offspring and is chosen to be 5 in this work, so in other words the population size is actually 6. the first step is to randomly generate a population of $1 + \lambda$ genotypes, then the fittest genotype is set as the parent. It gets promoted immediately to the next generation (step) and other members of population will be mutated versions of the parent. The mutation operator used in CGP is a point mutation operator. In a point mutation, an allele at a randomly chosen gene location is changed to another valid random value, i.e. with respecting the allelic constraints. The offspring of a higher or equal fitness of the parent is then set as the new parent, and so on [15].

4 Simulated Annealing

Simulated annealing (SA) is an optimization technique inspired by a process of heat treatment of materials that alters the properties of the material in a favorable way. It was proposed by several researchers [10, 11, 18] but one of the most recognized papers was published by Scott Kirkpatrick, Daniel C. Gelatt Jr. and Mario P. Vecchi in 1983 [12]. In this paper, the term “simulated annealing” was used as a name for this method for the first time and it also provided proofs of its efficiency by testing it on hard computational optimization problems (i.e. the Travelling Salesman Problem).

SA can be simply described as an iterative heuristic that generates a basic state s and then decides whether to move to a different state s^* . This decision is affected by several important factors with *acceptance probability function* $P(e, e_{next}, T)$ being the most important one. The outcome of this function depends on so called *energies* e and e_{next} of the actual and possible new state and also the *temperature* T which changes the sensitivity to energy variations. Initially the temperature is set to a high value, which allows for a broad

exploration of possible solutions. The temperature gradually decreases, which makes the solutions tend to lower energy states, and finally as the temperature approaches the point where $T = 0$, the algorithm stops. In some cases restarting the algorithm by increasing the temperature can provide increased solution variability.

5 NOVEL CGP-SA Algorithm

The innovation of this paper lies in the development of a modified Cartesian Genetic Programming algorithm, designated as CGP-SA. This variant incorporates a Boltzmann-type selection operator, a significant alteration designed to influence the rate of premature convergence commonly observed in genetic algorithms. By integrating an annealing temperature parameter ‘T’, a concept borrowed from simulated annealing techniques, the aim is to enhance the algorithm’s adaptability and efficiency in the search for global optima. This novel approach is expected to significantly improve the process of finding the most optimal circuit designs, offering a dynamic balance between exploration and exploitation in the evolutionary process. The incorporation of the temperature parameter is a strategic move to navigate the complex solution landscape more effectively, potentially leading to groundbreaking advancements in optimal circuit design through evolutionary algorithms.

ALGORITHM 1: CGP-SA Algorithm

```

init population  $1 + \lambda$  (choose set of gates);
init temperature  $T = T_{max}$ ;
init  $c = (0.95, 0.999)$ ;
compute fitness for all individuals;
the best becomes parent  $R$  and saved  $S$ ;
while not end condition do
    find best child  $N$ ;
    fitness difference  $\Delta f = fitness(N) - fitness(R)$ ;
     $u \in [0, 1]$  is a random number;
    if  $fitness(N) > fitness(S)$  then
        |  $S = N$ ;
    if  $\Delta f > 0 \vee exp(\Delta f/T) \geq u$  then
        |  $R = N$ ;
    create  $\lambda$  children by mutating the parent  $R$ ;
    new population consists of parent  $R$  and  $\lambda$  children;
    lower temperature  $T = c \cdot T$ ;
    if  $T < T_{min}$  then
        |  $T = T_{max}$ ;
end

```

6 Experiments and Results

The Benchmark problems for our improved method are chosen to be the 2×2 , 3×2 , and 3×3 multipliers. The aim here, as mentioned before, is to minimize the number of transistors used to build the electronic circuit. As the algorithm runs, it keeps track of the generation in which the first correct solution is found and when the best solution was found. For all problems, the following was chosen $\lambda = 5$ offspring, mutation rate 2% and $l = n_c$ (how far back can a gate reach for it's inputs). Each individual is scored by an objective function (1).

$$f = \begin{cases} b \cdot 100 & b < n_o 2^{n_i}, \\ b \cdot 100 + (100 - n_t/10) & \text{otherwise.} \end{cases} \quad (1)$$

where b is the number of matching bits in the truth table of the search circuit, n_t is the number of transistors in the circuit, n_i and n_o are the number of inputs and outputs. To test the effectiveness of CGP-SA on a given multiplier problem, different sets of gates (Table 2) and different CGP grid sizes (2D array of gates) were chosen. The $(not\ x)AND\ y$ gate is not a typical mass-produced gate, therefore it is counted as two gates (NOT, AND) in the presented results. The gate sets were chosen with consideration of the types of gates used in conventional circuit design.

Table 2. Used Sets of Gates

set 1:	$\Gamma = \{AND, NAND, OR, NOR, XOR, XNOR, (not\ x)AND\ y\}$
set 2:	$\Gamma = \{AND, OR, XOR\}$
set 3:	$\Gamma = \{NAND, NOR, XNOR\}$

Evolution was always started from a randomly generated population and 50 runs were performed for each setup. The termination condition was chosen based on the prior test runs and included algorithm stagnation and maximum number of generations. The temperatures for all runs were set to $T_{max} = 5$, $T_{min} = 0.1$, and $c = (0.95, 0.999)$.

The implementation of the evolutionary algorithm differs from the approaches described in [23] particularly in the approach to parent selection, where an approach inspired by simulated annealing was chosen. Another difference from traditional methods of optimizing combinational circuits using CGP is the choice of minimizing the number of transistors instead of the number of gates. It has been shown that circuits obtained in this way are equivalent in gate count to circuits developed in [21]. During the course of evolution, circuits with equivalent gate counts but different transistor counts have been found.

It also turned out that some gate sets are not suitable after the evolution of specific logic circuits, specifically **set 3**. The simpler 2×2 and 2×3 test circuits found are comparable to the results in [6, 17]. For the 3×3 multiplier, the

Table 3. Results obtained for different multipliers. The results are presented with gate counts (g.), transistor counts (t.) and circuit delay (d.).

	2×2 multiplier			3×2 multiplier			3×3 multiplier		
	g.	t.	d.	g.	t.	d.	g.	t.	d.
CGP-SA delay*	7	37	2	13	69	3	26	139	5
CGP-SA gates*	7	37	2	13	68	4	24	135	8
Petrlík, J., 2011 [17]	7	35	2	13	66	3	23	133	5
Sekanina, L., 2010 [6]	7	—	—	13	—	—	23	—	—

results of the chosen approach differ from these experiments. Since the primary optimization in this work was used to minimize transistors, the circuits found by this approach may have more gates for a comparable number of transistors. A comparison of results in delay and gate count is shown in Table 3.

6.1 2×2 Multiplier

The best 2×2 multiplier was selected as the individual from the evolutionary run with **set 1**; in the grid size 3×3 . This multiplier can be seen in Fig. 4. The terminating condition was set to 50 thousand generations.

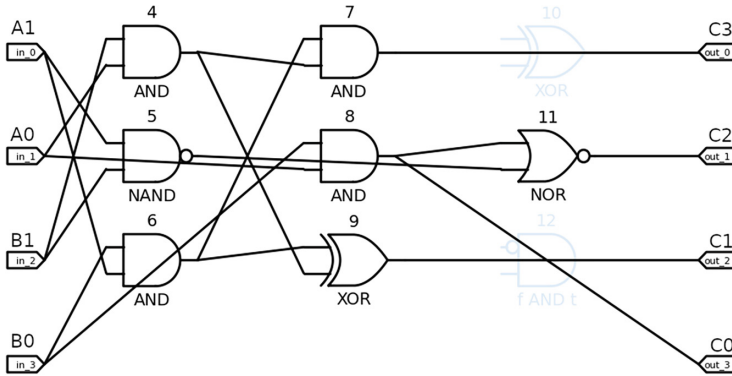


Fig. 4. 2×2 multiplier (7 gates, 37 transistors, 2T delay) in grid size 3×3 (shaded gates are unconnected)

6.2 3×2 Multiplier

The 3×2 multiplier performs multiplication of two binary numbers of three and two bits. From the results obtained, it was found that **set 2** finds the best results in all grid size settings. However, **set 1** is more suitable for transistor

minimization, because it contains “cheaper” gates. The best result across settings was found with **set 1** in the grid size 5×5 . This multiplier can be seen in Fig. 5. The terminating condition was set to 250 thousand generations.

6.3 3×3 Multiplier

The 1×35 grid was used to find the individual with the lowest number of gates. However, this circuit had a relatively high delay of 8T. Therefore, the best circuit was chosen to be the individual shown in Fig. 6, found in a 6×6 grid allowing usage of gates from **set 2**. This multiplier can be seen in Fig. 6. Although this individual does not have the lowest number of gates, it has only four more transistors and significantly better delay. Given the complexity of the problem, note that the termination condition was limited to 15 million generations.

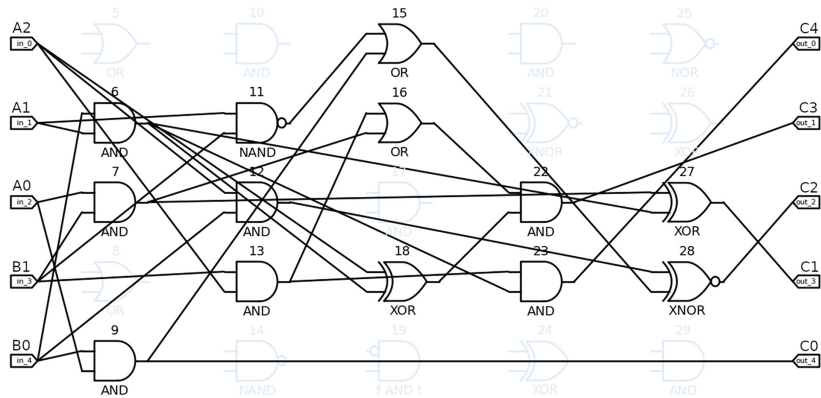


Fig. 5. 3×2 multiplier (13 gates, 69 transistors, 3T delay)

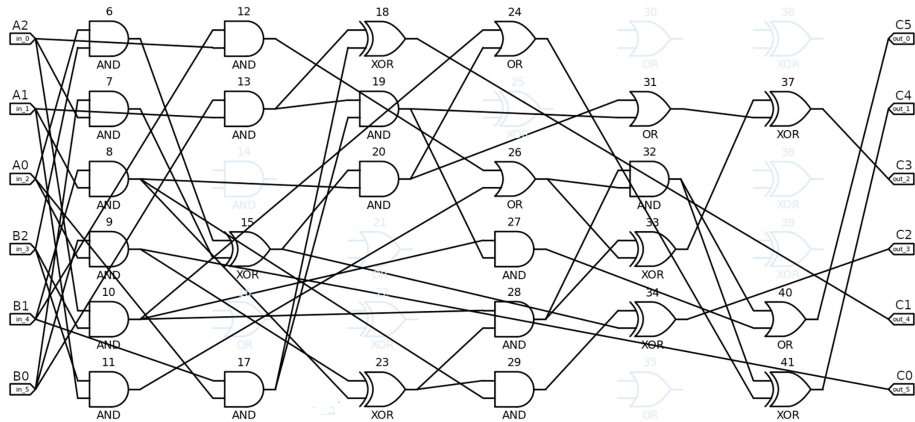


Fig. 6. 3×3 multiplier (26 gates, 139 transistors, 5T delay)

6.4 4×4 Full Adder

The 4×4 adder performs the sum of two four-bit numbers with a carry-in bit, resulting in 9 inputs and 5 outputs. In Table 4, we can see that the best average results are shown by using gates from **set 2**. However using gates from **set 1** several circuits with a smaller number of gates were found. The best circuit was chosen to be the one found with **set 1** in the 6×6 grid. This adder can be seen in Fig. 7. As a reference, the integrated circuit 74283 can be used, which has 36 gates and a 4T delay [5]. However, since this circuit uses multi-input gates, a direct comparison is not entirely ideal. Compared to our circuit it has significantly fewer gates, however their delay is longer. The terminating condition was set to 5 million generations.

Table 4. Results obtained for different grid configurations in CGP applied to a 4×4 full adder. The results are presented with gate counts (g.), transistor counts (t.) and circuit delay (d.).

Grid	set 1 best			set 2 best			set 1 avg			set 2 avg		
	g.	t.	d.	g.	t.	d.	g.	t.	d.	g.	t.	d.
[1,35]	20	120	8	20	120	8	21	142.3	10.4	20.3	129.7	9.3
[6,6]	23	147	6	25	140	6	25.3	158.3	6	26	146	2.6
[8,8]	21	136	6	22	128	7	24.1	162.5	7.4	23.6	141.9	7.5

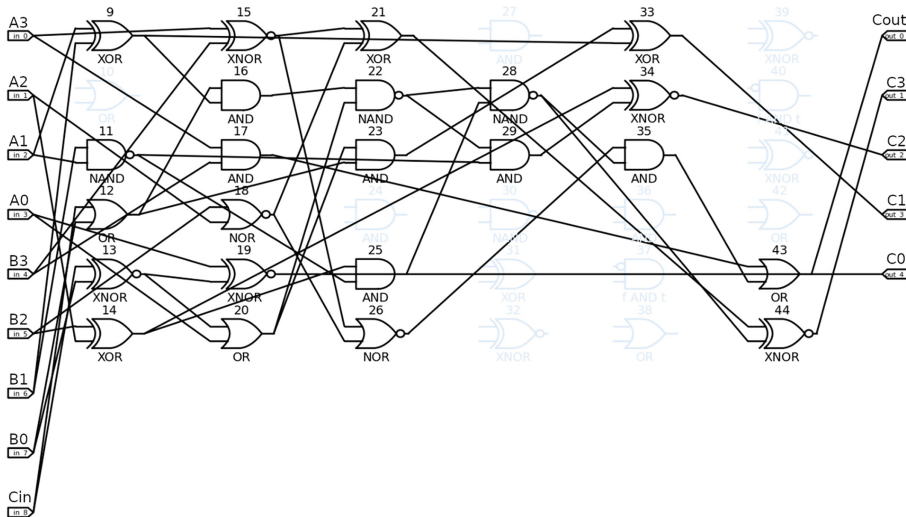


Fig. 7. 4×4 full adder (23 gates, 147 transistors, 6T delay)

7 Conclusion

This paper have described a modification of Cartesian Genetic Programming algorithm using Simulated Annealing for selection operator (CGP-SA). This innovative approach was rigorously tested across a variety of complex digital circuit design problems, including the synthesis of 2×2 , 3×2 , and 3×3 multipliers, as well as a 4×4 full adder. The results of these tests demonstrate the robustness of our implementation, successfully converging to efficient solutions that are not only comparable but in some cases superior to those achieved by other leading researchers in the field.

Notably, the CGP-SA method showed the ability find good trade-offs between gate count, transistor count, and overall circuit delay, particularly in challenging grid configurations. The results from the 6×6 grid, for instance, highlight the algorithm's capability to optimize circuit design with fewer gates, even with a slight increase in delay for some cases. This is especially evident when comparing the performance of the optimized circuits to conventional designs like the integrated circuit 74283 (4-bit binary adder).

Acknowledgment. This work was supported by Czech Science Foundation GACR 24-12474S: “Benchmarking derivative-free global optimization methods” and European COST Action CA22137: “Randomised Optimisation Algorithms Research Network (ROAR-NET)” and Internal grant agency of BUT FSI-S-23-8394 “AI methods in Engineering Applications”.

References

1. Baker, R.J.: CMOS: Circuit Design, Layout, and Simulation. Wiley, Hoboken (2019)
2. Davis, M., Liu, L., Elias, J.: VLSI circuit synthesis using a parallel genetic algorithm. In: Proceedings of the First IEEE Conference on Evolutionary Computation. IEEE World Congress on Computational Intelligence. IEEE (1994). <https://doi.org/10.1109/icec.1994.350033>
3. Dilek, S., Tosun, S., Cakin, A.: Simulated annealing-based high-level synthesis methodology for reliable and energy-aware application specific integrated circuit designs with multiple supply voltages. *Int. J. Circuit Theory Appl.* **51**(10), 4897–4938 (2023). <https://doi.org/10.1002/cta.3666>
4. Elgamal, M.: A genetic algorithm to optimize the performance of the tunneling field-effect transistor. *J. Comput. Electron.* **19**(3), 1068–1076 (2020). <https://doi.org/10.1007/s10825-020-01492-8>
5. Fairchild Semiconductor: 4-Bit Binary Adder with Fast Carry, August 1986. Revised March 2000
6. Gajda, Z., Sekanina, L.: An efficient selection strategy for digital circuit evolution. In: Tempesti, G., Tyrrell, A.M., Miller, J.F. (eds.) *Evolvable Systems: From Biology to Hardware*, pp. 13–24. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15323-5_2

7. Kalganova, T., Miller, J.: Evolving more efficient digital circuits by allowing circuit layout evolution and multi-objective fitness. In: Proceedings of the First NASA/DoD Workshop on Evolvable Hardware, pp. 54–63 (1999). <https://doi.org/10.1109/EH.1999.785435>
8. Kang, J.Y., Gaudiot, J.L.: A simple high-speed multiplier design. *IEEE Trans. Comput.* **55**(10), 1253–1258 (2006). <https://doi.org/10.1109/tc.2006.156>
9. Karimullah, S., Vishnuvardhan, D., Arif, M., Gunjan, V.K., Shaik, F., Siddiquee, K.N.e.a.: An improved harmony search approach for block placement for VLSI design automation. *Wirel. Commun. Mobile Comput.* **2022**, 3016709 (2022). <https://doi.org/10.1155/2022/3016709>
10. Khachaturyan, A., Semenovskaya, S., Vainstein, B.: Statistical-thermodynamic approach to determination of structure amplitude phases. *Sov. Phys. Crystallography* **24**(5), 519–524 (1979)
11. Khachaturyan, A., Semenovskaya, S., Vainshtein, B.: The thermodynamic approach to the structure analysis of crystals. *Acta Crystallographica Sect. A Cryst. Phys. Diffract. Theoret. General Crystallogr.* **37**(5), 742–754 (1981)
12. Kirkpatrick, S., Gelatt, C.D., Jr., Vecchi, M.P.: Optimization by simulated annealing. *Science* **220**(4598), 671–680 (1983)
13. Meitei, N.Y., Baishnab, K.L., Trivedi, G.: 3D-IC partitioning method based on genetic algorithm. *IET Circuits Devices Syst.* **14**(7), 1104–1109 (2020). <https://doi.org/10.1049/iet-cds.2020.0128>
14. Miller, J.: PPSN 2014 tutorial: cartesian genetic programming (2014). <http://ppsn2014.ijs.si/files/slides/ppsn2014-tutorial3-miller.pdf>
15. Miller, J.F. (ed.): *Cartesian Genetic Programming*. Natural Computing Series. Springer, New York (2011)
16. Miller, J.F., Job, D., Vassiliev, V.K.: Principles in the evolutionary design of digital circuits-part I. *Genet. Program. Evolvable Mach.* **1**, 7–35 (2000). <https://doi.org/10.1023/A:1010016313373>
17. Petrlik, J.: *Multikriteriální kartézské genetické programování*. Ph.D. thesis, Brno University of Technology (2011). <http://hdl.handle.net/11012/54080>
18. Pincus, M.: A Monte Carlo method for the approximate solution of certain types of constrained optimization problems. *Oper. Res.* **18**(6), 1225–1228 (1970)
19. Reis, C., Machado, J.A.T.: *Particle Swarm Design of Digital Circuits*, pp. 333–343. Springer, Dordrecht (2009). https://doi.org/10.1007/978-1-4020-8678-6_28
20. Schafer, B.C., Wang, Z.: High-level synthesis design space exploration: past, present, and future. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* **39**(10), 2628–2639 (2020). <https://doi.org/10.1109/tcad.2019.2943570>
21. Sekanina, L.: *Evolvable Components*. Springer, Heidelberg (2004). <https://doi.org/10.1007/978-3-642-18609-7>
22. Sekanina, L., Ptak, O., Vasicek, Z.: Cartesian genetic programming as local optimizer of logic networks. In: *IEEE Congress on Evolutionary Computation (CEC)*, 6–11 July 2014, pp. 2901–2908 (2014)
23. Sekanina, L., Walker, J.A., Kaufmann, P., Platzner, M.: *Evolution of Electronic Circuits*, pp. 125–179. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-17310-3_5
24. Vu, H.G., Bui, N.D., Nguyen, A.T., ThanhBangLe: Performance evaluation of quine-McCluskey method on multi-core CPU. In: *2021 8th NAFOSTED Conference on Information and Computer Science (NICS)*. IEEE, December 2021. <https://doi.org/10.1109/nics54270.2021.9701506>
25. Wakerly, J.F.: *Digital Design: Principals & Practices*. Prentice Hall, Upper Saddle River (2000)

26. Wang, Y., DeBrunner, L.S., DeBrunner, V.E., Zhou, D.: A multi-input multiplier unit suitable for DSP algorithm implementations. In: 2006 Fortieth Asilomar Conference on Signals, Systems and Computers, pp. 1465–1468, December 2006. <https://doi.org/10.1109/ACSSC.2006.355001>
27. Wu, T., He, J.: Differential evolution with group crossover for automatic synthesis of analog circuit. In: Tan, Y., Shi, Y., Mo, H. (eds.) *Advances in Swarm Intelligence*, pp. 414–421. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-38703-6_49
28. Zhang, Z., Li, S., Wang, X., Cheng, W., Qi, Y.: Source mask optimization for extreme-ultraviolet lithography based on thick mask model and social learning particle swarm optimization algorithm. *Opt. Express* **29**(4), 5448–5465 (2021). <https://doi.org/10.1364/OE.418242>